

R213 TP 1: Prise en main SciLab, IF/FOR/WHILE

Exercice 1: Trouver ce que font les instructions suivantes, puis tapez ces lignes dans *SciNotes*. Ceci constitue un programme. Testez-le avec divers valeurs de x comme 0, -4, 9, π^2 , $(-\pi)^2$, ...

```
1 x=input('x?');
2 if x>0 then
3     disp('racine(x)=')
4     disp(sqrt(x))
5 end;
```

Si $x > 0$, il affiche "racine(x)=" suivi de sqrt(x), c'est-à-dire la racine carrée de x. Exemple : quand on écrit 49 il nous donne 7 car $7 \times 7 = 49$ ou pour 16 il nous donne 4 car $4 \times 4 = 16$

Exercice 2: Le programme suivant simule le tirage au hasard avec équiprobabilité d'une boule dans une urne qui contient quatre boules blanches et une noire. Tapez le et testez le.

```
1 rand('uniform');
2 pioche=rand();
3 if pioche < 0.2 then
4     disp('noire')
5 else
6     disp('blanche')
7 end
```

Le programme a deux valeurs possibles, soit le programme tire 'noire' avec 20% de chance soit il tire 'blanche' avec 80% de chance.

Exercice 3: Trouver ce que font les instructions suivantes, puis taper le script et le tester avec diverses valeurs de x et y.

```
1 x=input('x?');
2 y=input('y?');
3 if x>y then
4     disp(x)
5 else
6     disp(y)
7 end;
```

Le programme demande a donné des valeur pour X et pour Y, si la valeur qu'on donne pour x est plus grand que celui pour y ça nous affiche la valeur de x si non la valeur de y.

Exercice 4: Ecrire un programme qui affiche « oui » ou « non » selon qu'un entier n est divisible par 7 ou non. Testez-le avec plusieurs valeurs.

```
1 n=input("La valeur.-.")
2 if modulo(n,7)==0 then
3     ....disp("oui")
4 else
5     ....disp("non")
6 end
```

Le programme demande une valeur et fait le modulo de 7 de la valeur donnée. Si le modulo fonctionne, il répond oui sinon il répond non. Voici quelques exemples :

pour $n = 7$

```
--> exec('C:\Users\22402312\Documents\ex8 r114.sce', -1)
La valeur =
7
```

pour $n = 3$

```
--> exec('C:\Users\22402312\Documents\ex8 r114.sce', -1)
La valeur =
3
```

Exercice 5: Ecrire un programme qui demande trois nombres réels a, b, c avec $a \neq 0$ puis résout l'équation $E: ax^2 + bx + c = 0$ d'inconnue $x \in \mathbb{R}$

```
a = input("Entrez a:");
b = input("Entrez b:");
c = input("Entrez c:");

Delta = b^2 - 4*a*c;

x = [(-b - sqrt(Delta)*%i) / (2*a), (-b + sqrt(Delta)*%i) / (2*a)];
1
disp("Les solutions sont: ", disp(x))
```

```
Entrez a :
1
Entrez b :
-3
Entrez c :
2

Les solutions sont :

1.5 - 0.5i 1.5 + 0.5i
exec : Nombre erroné d'argument(s) de sortie
```

Exercice 6 : Trouver ce que font les instructions suivantes puis vérifier en tapant le script.

```
1 for n=0:5
2     ....disp(n*n)
3 end
```

Le programme va parcourir toutes les valeurs entre 0 et 5 donc 0,1,2,3,4 et 5 puis il multiplie chaque valeur par elle-même donc on obtient :

```
--> exec('C:\Users\22402312\Documents\ex8 r114.sce', -1)

0.
1.
4.
9.
16.
25.
```

$0 \times 0 = 0$
 $1 \times 1 = 1$
 $2 \times 2 = 4$
 $3 \times 3 = 9$
 $4 \times 4 = 16$
 $5 \times 5 = 25$

Exercice 7: Trouvez ce que fait la fonction suivante, puis testez-le.

```
1 function y=factorielle(n)
2     f=1; //0!=1
3     for k=1:n
4         f=f*k;
5     end
6     y=f;
7 endfunction
```

Ce programme permet de trouver la factorielle d'une valeur n.
Par exemples :

```
--> factorielle(3)
ans =

    6.
```

La factorielle de 3 donc $1 \times 2 \times 3 = 6$

```
--> factorielle(4)
ans =

    24.
```

La factorielle de 4 donc $1 \times 2 \times 3 \times 4 = 24$

```
--> factorielle(5)
ans =

   120.
```

La factorielle de 5 donc $1 \times 2 \times 3 \times 4 \times 5 = 120$

Exercice 8: Ecrire un programme qui calcule et affiche la valeur de $1 + \frac{1}{1,2} + \frac{1}{1,4} + \dots + \frac{1}{1,8} + \frac{1}{2}$.

On utilisera une variable S traditionnellement appelée accumulateur dans laquelle on va mettre successivement les valeurs des sommes partielles : $1; 1 + \frac{1}{1,2}; 1 + \frac{1}{1,2} + \frac{1}{1,4}; \dots$

```
R=1;
for x=1.2:0.2:1.8
    R=R+1/x;
end
R=R+1/2;
disp("Valeur de S =", R)
```

```
Attention : Le fichier 'C:\Users\22402312\
Valeur de S =

    4.2281746
```

explication:

- R = 1 correspond au premier terme de la série.
- Boucle for : dans laquelle on peut générer les valeurs 1.2, 1.4, 1.6, 1.8, 2.0
- On ajoute chaque $1/x$ à S. pour avoir la valeur suivante de notre boucle
- Affichage du résultat final ce qui nous affiche la valeur de $S = 4.22.81746$

Exercice 9:

En l'an 2022, le lycée A compte 2 000 élèves et le lycée B compte 8 000 élèves.

Une étude montre que, chaque année :

- 10 % des élèves du lycée A quittent le lycée A pour aller au lycée B ;
- 15 % des élèves du lycée B quittent le lycée B pour aller au lycée A.

1) Combien y aura-t-il d'élèves au lycée A et au lycée B en 2023 ? en 2024 ?

2) Faire un programme qui calcule la population des deux lycées sur 20 ans et donner une représentation graphique de cette situation.

- 1) en 2023 le lycée A $\Rightarrow$ 3000 élève
en 2023 le lycée B $\Rightarrow$ 7000 élève

```
--> A2 = 2000-0.1*3000+0.15*8000
A2 =
    2900.

--> A2 = 2000-0.1*2000+0.15*8000
A2 =
    3000.

--> 2000-200+1200
ans =
    3000.

--> |
```

```
--> 8000-0.15*8000+0.1*2000
ans =
    7000.

--> 8000-1200+200
ans =
    7000.
```

- en 2024 le lycée A $\Rightarrow$ 3750 élèves
en 2024 le lycée B $\Rightarrow$ 6250 élèves

```
--> 3000-0.1*3000+0.15*7000
ans =
    3750.

--> 3000-300+1050
ans =
    3750.
```

```
--> 7000-0.15*7000+0.1*3000
ans =
    6250.

--> 7000-1050+300
ans =
    6250.
```

- 2) explication de programme

On doit définir les effectifs initiaux:

- 2000 élève à A et 8000 élèves à B en 2022
- $n == 20 \Rightarrow$ on simule l'évolution sur 20 ans.
- An et Bn sont donc les vecteurs où on stock les effectifs années par année

```

A = 2000;
B = 8000;
N = 20;

An = zeros(1, N+1);
Bn = zeros(1, N+1);

An(1) = A;
Bn(1) = B;

for n = 1:N
    A_new = A - 0.1*A + 0.15*B;
    B_new = B - 0.15*B + 0.1*A;
    An(n+1) = A_new;
    Bn(n+1) = B_new;
    A = A_new;
    B = B_new;
end

disp("Année...Lycée A...Lycée B")
for n = 0:N
    disp([2022+n, An(n+1), Bn(n+1)])
end

```

```

--> exec('C:\Users\22402312\scnotevfodvd.sce', -1)

Année    Lycée A    Lycée B
2022.    2000.    8000.
2023.    3000.    7000.
2024.    3750.    6250.
2025.    4312.5   5687.5
2026.    4734.375 5265.625
2027.    5050.7813 4949.2188
2028.    5288.0859 4711.9141
2029.    5466.0645 4533.9355
2030.    5599.5483 4400.4517
2031.    5699.6613 4300.3387
2032.    5774.7459 4225.2541
2033.    5831.0595 4168.9405
2034.    5873.2946 4126.7054
2035.    5904.9709 4095.0291
2036.    5928.7282 4071.2718
2037.    5946.5462 4053.4538
2038.    5959.9096 4040.0904
2039.    5969.9322 4030.0678
2040.    5977.4492 4022.5508
2041.    5983.0869 4016.9131
2042.    5987.3152 4012.6848

```

Exercice 10: Trouver ce que font les instructions suivantes puis vérifier en tapant le script.

```

1 x=input('tapez un nombre.>0');
2 while (x<=0)
3     disp('vous n\'avez pas tape un nombre.>0');
4     x=input('tapez un nombre.>0')
5 end
6 disp('bravo')

```

Ce programme est une boucle while à condition : l'utilisateur doit donner un nombre positif et tant qu'il n'aura pas donné un nombre positif, la boucle continuera.

Exercice 11: Soit (u_n) la suite géométrique de premier terme $u_0 = 400$ et de raison $q = 0,6$. On pose pour tout entier naturel n , $S_n = u_0 + u_1 + \dots + u_n$.

- Quelle est la limite de la suite (S_n) ?
- Ecrire une fonction appelé **somme** qui affiche le terme de rang n de la suite (S_n) .
- Conjecturer à l'aide de *ScilaB* la limite de la suite (S_n) ?
- Ecrire une fonction seuil qui retourne le rang n à partir duquel $S_n > 998$. On pourra utiliser la fonction **somme**.

a)

Pour trouver la limite de notre suite géométrique S_n on peut utiliser la formule du cours :

$$U_0 \times (1 - q^{n+1}) / (1 - q)$$

Puis on pousse progressivement la valeur de n vers $+\infty$

On obtient donc : $\lim_{n \rightarrow \infty} S_n = 400 / (1 - 0,6) = 400 / 0,4 = 1000$

La limite tend vers 1000

b)

```
1 function S = somme(n)
2   ... u0=400
3   ... q=0.6
4   ... S=u0 * (1 - q^(n+1)) / (1-q);
5 endfunction
```

Le programme demande une valeur à l'utilisateur (la valeur n) qui sera donc le terme de rang n , définit le premier terme et la raison, puis applique la formule du cours avec les valeurs données.

c)

Valeur de n	0,5	1	3	5	10	50	100
Résultats	535.242	640	870.4	953.344	996.37203	1000	1000

d)

```
1 function n = seuil()
2   ... n = 0;
3   ... while somme(n) <= 998
4   ...     n = n + 1;
5   ... end
6 endfunction
```

Le programme fait défiler toutes les sommes à partir de 0 jusqu'à temps que la valeur soit supérieure à 998.

```
--> seuil()
ans =
```

Notre valeur est supérieure à 998 à partir du rang 12.

12.

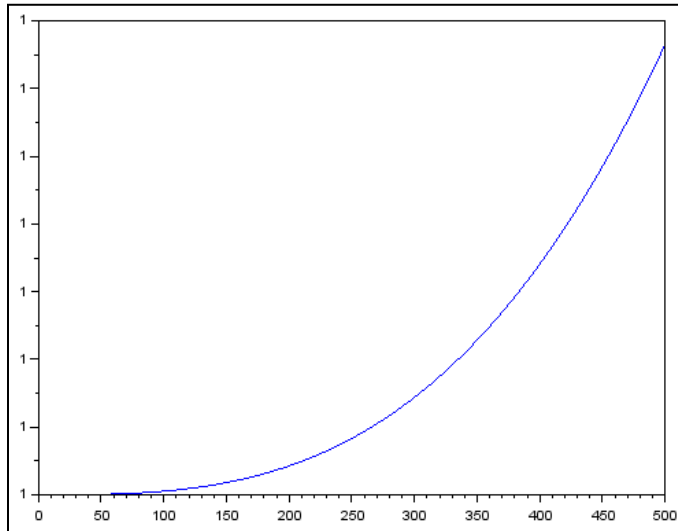
Exercice 12:

On considère la suite (V_n) définie pour tout entier naturel non nul n par:

$$v_n = 1 + \frac{1}{2^2} + \frac{1}{3^2} + \dots + \frac{1}{n^2}$$

Calculer 500 termes de cette suite et les représenter graphiquement.

Rechercher un entier n_0 tel que pour $n \geq n_0$, on ait $v_{n+1} - v_n \leq 10^{-5}$.



```
--> n = 1:500;  
--> V = cumsum(1 / (n^2));  
--> V = 1 + V;
```

On définit les valeurs pour l'abscisse :

De 1 à 500

On définit les valeurs pour l'ordonnée :

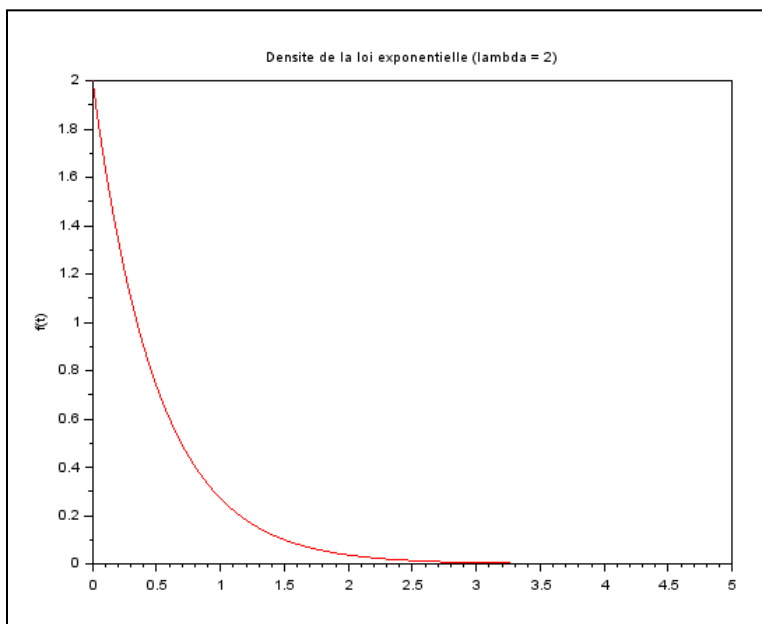
Somme cumulée de $1/n^2$

Puis on définit le premier terme (1)

et on affiche (plot(x, y))

Exercice 13: Ecrire une fonction expo(lambda, t) qui donne en sortie une densité de la loi exponentielle de

paramètre λ : $\text{expo}(\lambda, t) = \begin{cases} 0 & \text{si } t \leq 0 \\ \lambda e^{-\lambda t} & \text{si } t > 0 \end{cases}$



```
--> t = 0:0.01:5;  
--> y = expo(2, t);  
--> plot(t, y, 'r')  
--> ylabel("f(t)")  
--> title("Densite de la loi exponentielle (lambda = 2) ")
```

```
function f = expo(lambda, t)  
    if t <= 0 then  
        f = 0;  
    else  
        f = lambda * exp(-lambda * t);  
    end  
endfunction  
disp(expo(2, -1))  
disp(expo(2, 0))  
disp(expo(2, 1))
```

Exercice 14 : Reprendre l'exercice 9 et écrire un programme qui calcule l'année à partir de laquelle le lycée A comptera plus d'élèves que le lycée B.

```
Annee = 2022;
A = 2000;
B = 8000;

while A <= B
    Nouveaux_A = A + 0.15 * B - 0.10 * A;
    Nouveaux_B = B + 0.10 * A - 0.15 * B;

    A = Nouveaux_A;
    B = Nouveaux_B;
    Annee = Annee + 1;
end

disp("Le lycée A dépassera le lycée B en 1 annee : " + string(Annee));
```

Dans le programme , on initialise les effectifs en 2022.

- La boucle while tourne tant que $A \leq B$.
- À chaque itération :
- On applique les transferts (10 % de A vers B et 15 % de B vers A).
- On met à jour A et B.
- On incrémente l'année.
- La boucle s'arrête quand $A > B$, et on affiche l'année correspondante au quelle le lycée A dépassera le lycée B en L'année $\Rightarrow 2027$

```
--> exec('C:\Users\22402312\scnotevfodvd.sce', -1)

Le lycée A dépassera le lycée B en 1 annee : 2027
```

Exercice 15: Algorithme de Syracuse

On choisit un entier naturel quelconque. S'il est pair, on le divise par 2, sinon on le multiplie par 3 et on ajoute 1. En répétant ce processus, on constate qu'au bout d'un certain nombre d'itérations, on aboutit à 1 (actuellement personne n'a su le démontrer).

Exemple : 7 ; 22 ; 11 ; 34 ; 17 ; 52 ; 26 ; 13. ...

- Terminer « à la main » la liste précédente.
- Écrire un programme syracuse(n) qui affiche le nombre suivant le nombre n dans la liste de syracuse.
- Compléter ce programme de façon à lui faire effectuer la boucle complète jusqu'à arriver à 1.

Exercice 16: Ecrire un programme qui donne la décomposition en facteurs premiers d'un nombre entier naturel supérieur ou égal à 2.

Explication de programme:

On doit refuser tous les valeurs < 2

dans notre boucle principale on divise n par les facteurs 1er successifs (2, 3, 5, 7, ...)

puis on doit afficher chaque facteur avec son exposant mais c'est pas le cas tout le temps donc si nécessaire

et à la fin notre programme doit s'arrêter une fois que $n = 1$

```
function facteur(n)
--if n < 2 then
-----disp("Le nombre doit être supérieur ou égal à 2");
-----return;
--end

--disp("Décomposition en facteurs premiers de " + string(n) + " :");
--
facteur = 2;
first = %F;

--while n > 1
-----count = 0;
-----
-----while modulo(n, facteur) == 0
-----    n = n / facteur;
-----    count = count + 1;
-----end
-----
-----if count > 0 then
-----    if first then
-----        first = %F;
-----    else
-----        mprintf(" * ");
-----    end
-----    mprintf("%d", facteur);
-----    if count > 1 then
-----        mprintf("^%d", count);
-----    end
-----end
-----facteur = facteur + 1;
--end

--mprintf("\n");
endfunction
```

```
--> facteur(60)

Décomposition en facteurs premiers de 60 :
2^2 * 3 * 5

--> facteur(90)

Décomposition en facteurs premiers de 90 :
2 * 3^2 * 5

--> facteur(69)

Décomposition en facteurs premiers de 69 :
3 * 23
```